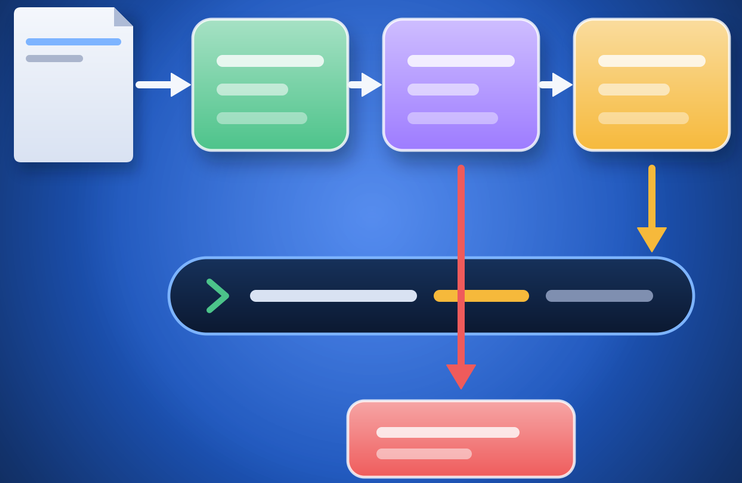
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 4

Redirection and Pipelines

Lab 4 guide: send output and errors to different places, connect commands, and build a report you can trust.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Use redirection and pipelines, and tell stdout, stderr and the exit status apart.
2. Select records, extract fields and count matches in a CSV file.
3. Test exact matching, changed inputs and missing files.

THIS LAB

Lab 4

First the concepts, with examples. Then the lab on the server, where you work directly at the terminal. Watching and signalling processes are optional extensions, so they come at the end.

DISCUSSION

A command prints an error and a result. Where does each go?

Both appear on your screen. Are they the same thing? How would you save only one of them?

CONCEPTS FIRST

Streams, pipes and filters

Three ideas: every program has three streams, a pipe joins two programs, and small filters do one job each.

Every command has three streams

Stream	Number	Default	Used for
stdin	0	The keyboard	Input
stdout	1	The terminal	Normal output
stderr	2	The terminal	Error messages and diagnostics

Both output streams appear on the screen, so they look the same. They are different streams, and you can send them to different places.

The redirection operators

Operator	Meaning	Example
`>`	Send stdout to a file, replacing it	`ls > files.txt`
`>>`	Append stdout to a file	`echo done >> log.txt`
`2>`	Send stderr to a file	`ls /fake 2> errors.txt`
`2>&1`	Make stderr go where stdout goes now	`cat example-events.csv missing-demo.txt > all.txt 2>&1`
`&>`	Bash shorthand: stdout and stderr into one file	`cat example-events.csv missing-demo.txt &> all.txt`
`<`	Read stdin from a file	`sort < unsorted.txt`
`/dev/null`	Throw the output away	`ls /fake > /dev/null 2>&1`

A single > replaces the file. Use >> to add to it. In a pipeline, |& sends both streams to the next command.

Two streams, two files

Example

student@server: ~/oslab-work/lab4

```
$ printf 'normal output\n' > evidence/stdout-demo.txt
$ cat missing-demo.txt 2> evidence/stderr-demo.txt
$ cat evidence/stdout-demo.txt
normal output
$ cat evidence/stderr-demo.txt
cat: missing-demo.txt: No such file or directory
```

printf writes the text. > sends it into the first file. The second command fails on purpose, and 2> catches only its error message.

Nothing about the error appeared on the screen the first time. It went into the file.

The order of redirections matters

Example 1: stderr follows stdout to the file

student@server: ~/oslab-work/lab4

```
$ printf 'ready,4\n' > example-events.csv
$ cat example-events.csv missing-demo.txt > both.txt 2>&1
$ cat both.txt
ready,4
cat: missing-demo.txt: No such file or directory
```

First stdout goes to the file. Then 2>&1 points stderr at that same place.

Example 2: stderr stays on the screen

student@server: ~/oslab-work/lab4

```
$ cat example-events.csv missing-demo.txt 2>&1 > only-out.txt
cat: missing-demo.txt: No such file or directory
$ cat only-out.txt
ready,4
```

Here stderr is pointed at stdout while stdout is still the screen. Only then does stdout move to the file.

Run Example 1 before Example 2. Read redirections from left to right.

A pipe joins two commands

- The pipe | connects the stdout of one command to the stdin of the next.
- The commands run at the same time. Data flows through as it is produced.
- Only stdout goes through the pipe. stderr still goes to the terminal.
- The exit status of a pipeline is, by default, the status of the last command.

Example

student@server: ~/oslab-work/lab4

```
$ printf 'ok,3\nfail,2\n'
ok,3
fail,2
$ printf 'ok,3\nfail,2\n' | grep '^ok,'
ok,3
$ printf 'ok,3\nfail,2\n' | grep '^ok,' | cut -d, -f2
3
```

Read it stage by stage: all rows, then only the ok rows, then only field 2 of those rows.

EXAMPLE

Try it on a sample CSV

Use these sample files to see each command's input and output before opening your personal lab.

Write, append and read

Example

student@server: ~

```
$ mkdir -p ~/lab4-examples
$ cd ~/lab4-examples
$ printf 'status,count\nready,4\n' > events.csv
$ printf 'done,2\nready-late,9\nready,7\n' >> events.csv
$ cat events.csv
status,count
ready,4
done,2
ready-late,9
ready,7
```

- mkdir -p creates the practice folder. cd enters it.
- printf writes text. The escape sequence creates a new line.
- Use > to replace the file and >> to append. cat displays the whole file.

grep selects matching lines

Example

student@server: ~/lab4-examples

```
$ grep '^ready,' events.csv
ready,4
ready,7
```

- Syntax: grep [OPTIONS] 'PATTERN' [FILE ...]. With no file, grep reads stdin.
- grep uses basic regex by default. Single quotes protect the pattern from shell expansion.
- Each match is printed as a complete line.
- The caret (^) means start of line. The comma ends the first field, so ready-late is skipped.

Common grep options

Option	What it does	Example
-i	Ignore case	`grep -i '^READY,' events.csv`
-n	Show the line number with each match	`grep -n '^ready,' events.csv`
-v	Select lines that do not match	`grep -v '^status,' events.csv`
-c	Print the number of selected lines per file	`grep -c '^ready,' events.csv`
-o	Print only the matched parts, one per line	`grep -o '[[:digit:]]$' events.csv`
-x	Require a match across the whole line	`grep -x 'ready,4' events.csv`
-E	Use extended regular expressions	`grep -E '^(ready|done),' events.csv`
-F	Treat the pattern as literal text	`grep -F 'ready-late,' events.csv`

Number, exclude and count matches

Example

student@server: ~/lab4-examples

```
$ grep -in '^READY,' events.csv
2:ready,4
5:ready,7
$ grep -v '^status,' events.csv
ready,4
done,2
ready-late,9
ready,7
$ grep -c '^ready,' events.csv
2
```

- Combine short options: -in is the same as -i -n.
- -v keeps the data rows by excluding the header.
- -c counts matching lines. Multiple matches on one line still contribute one line.

Regex reference: anchors and characters

Syntax	What it matches	Pattern	Matches in the sample
^	Start of the line	`^ready,`	ready,4 and ready,7
$	End of the line	`,7$`	ready,7
.	Any one character on the line	`^d.ne,`	done,2
[abc]	One of the listed characters	`,[47]$`	ready,4 and ready,7
[a-z]	One character in a range	`^[d-f]`	done,2
[^abc]	One character outside the set	`,[^47]$`	done,2 and ready-late,9
*****	Zero or more of the preceding item	`^re*ady,`	ready,4 and ready,7
\.	A literal dot	`\.txt$`	lab4.txt in a filename list

Regex reference: groups and repetition

Syntax with grep -E	What it does	Example pattern
`(...)`	Group a sequence into one item	`^(ready|done),`
`|`	Match either alternative	`^(ready|done),`
`+`	Repeat the preceding item one or more times	`[[:digit:]]+$`
`?`	Repeat the preceding item zero or one time	`^ready(-late)?,`
`{n}`	Repeat exactly n times	`[[:digit:]]{2}$`
`{n,}`	Repeat at least n times	`[[:digit:]]{2,}$`
`{n,m}`	Repeat between n and m times	`[[:digit:]]{1,3}$`

Use grep -E with the patterns on this slide. Add ^ or $ when the position of a match matters.

Regex reference: special character classes

Class or boundary	What it matches	Example
`[[:digit:]]`	A digit	`,[[:digit:]]$`
`[[:alpha:]]`	A letter	`^[[:alpha:]]`
`[[:alnum:]]`	A letter or digit	`[[:alnum:]]$`
`[[:lower:]] / [[:upper:]]`	A lowercase or uppercase letter	`^[[:upper:]]`
`[[:space:]] / [[:blank:]]`	Whitespace / horizontal blank	`^[[:space:]]*$`
`[[:punct:]]`	A punctuation character	`[[:punct:]]`
`\b`	A word boundary in GNU grep	`\bready\b`

Keep both pairs of brackets in [[:digit:]]. A hyphen creates a word boundary, so a word match for ready also finds ready-late.

Use regex and literal patterns

Example

student@server: ~/lab4-examples

```
$ grep -E '^(ready|done),' events.csv
ready,4
done,2
ready,7
$ grep -o '[[:digit:]]$' events.csv
4
2
9
7
$ printf 'lab4.txt\nlab4Xtxt\n' | grep '\.txt$'
lab4.txt
$ printf 'lab4.txt\nlab4Xtxt\n' | grep -Fx 'lab4.txt'
lab4.txt
```

- -E lets the parenthesized alternatives share the start anchor and trailing comma.
- -o shows just the last digit on each matching line.
- Escape the dot with a backslash in a regex. Use -Fx when you already know the exact whole line.

cut syntax and field lists

Syntax	What it does	Example
`-d DELIM`	Set the input field separator. The default is a tab.	`cut -d ',' -f 2 events.csv`
`-f LIST`	Select field numbers starting at 1	`cut -d, -f2 events.csv`
`-f1,3`	Select fields 1 and 3	`cut -d, -f1,3 people.txt`
`-f2-3`	Select fields 2 through 3	`cut -d, -f2-3 people.txt`
`-f2-`	Select from field 2 to the end	`cut -d, -f2- people.txt`
`-s`	Skip lines with no delimiter in field mode	`cut -s -d, -f2 people.txt`
`-c LIST`	Select character positions in ASCII text	`cut -c1-3 people.txt`

Syntax: cut -d DELIM -f LIST [FILE ...] or cut -c LIST [FILE ...]. Omit the file to read stdin. Use field mode for simple delimited text.

cut extracts a field

Example

student@server: ~/lab4-examples

```
$ cut -d, -f1 events.csv
status
ready
done
ready-late
ready
$ grep '^ready,' events.csv | cut -d, -f2
4
7
```

- Use -d, for comma-separated fields and -f1 for the first field. cut also reads the header.
- In the pipeline, grep selects the ready records first. Then -f2 extracts their quantities.

Select multiple fields or character positions

Example

student@server: ~/lab4-examples

```
$ printf '101,Ada,KH\n102,Lina,SG\n' > people.txt
$ cut -d, -f1,3 people.txt
101,KH
102,SG
$ cut -d, -f2-3 people.txt
Ada,KH
Lina,SG
$ cut -c1-3 people.txt
101
102
$ printf '101,Ada,KH\nmissing\n' | cut -s -d, -f2
Ada
```

- A comma in the field list selects individual fields. A hyphen selects a range.
- Here -c1-3 takes the first three character positions in each line.
- -s skips the line missing because it contains no comma.

wc counts the selected lines

Example

student@server: ~/lab4-examples

```
$ grep '^ready,' events.csv | wc -l
2
```

- wc -l counts lines received through the pipe. Each selected line is one record.
- The quantities 4 and 7 are fields on those two lines.

Display, save and compare

Example

student@server: ~/lab4-examples

```
$ grep '^ready,' events.csv | tee selected.txt
ready,4
ready,7
$ printf 'done,2\n' | tee -a selected.txt
done,2
$ cp -- selected.txt copy.txt
$ diff -u selected.txt copy.txt
```

- tee displays the selected rows and saves them. tee -a also keeps the existing lines.
- cp makes a copy. diff -u prints nothing when the files match.

Save diagnostics and inspect the exit status

Example

student@server: ~/lab4-examples

```
$ cat missing.csv > normal.txt 2> errors.txt
$ echo $?
1
$ cat errors.txt
cat: missing.csv: No such file or directory
$ cat events.csv missing.csv > combined.txt 2>&1
$ cat missing.csv | wc -l
cat: missing.csv: No such file or directory
0
$ echo $?
0
```

- The first cat leaves normal.txt empty and writes its diagnostic to errors.txt.
- The combined file receives both streams. In the pipeline, wc succeeds on empty input.

Check your Lab 4 workspace

Example

student@server: ~/oslab-work/lab4

```
$ oslab doctor
$ oslab prelab lab4
$ oslab start lab4
$ oslab values lab4
status = ready
other  = done
$ oslab hint lab4 1
$ oslab check lab4
```

- doctor checks setup. prelab opens the short review.
- start prepares your folder. values prints your own status words.
- hint gives a pointer. check reports the saved evidence milestones.

LAB 4

Read the help-desk event log

Select records, extract their quantities, and save normal output and diagnostics.

Lab 4: Redirection and Pipelines

- Work directly in the terminal. No Bash script is required.
- Goal: select records, extract fields, count lines, and distinguish normal output from errors.
- 120 minutes • individual account • personal status words

Read the exit status

- Run echo $? immediately after the command you want to check.
- Status 0 indicates success. For grep, 1 means no match and 2 means an error.
- A pipeline normally reports the status of its last command.

Setup: 0–10 minutes

Example

student@server: ~/oslab-work/lab4

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab4
$ cd "$OSLAB_WORKSPACE/lab4"
$ mkdir -p evidence
$ oslab values lab4
$ cat data/events.csv
```

- Replace STATUS and OTHER with your own words.
- Never reset to fix one command.
- For status ready, type grep '^ready,' to exclude ready-late.

GUIDED

Streams, append and tee

Run each line in the terminal and inspect what it produces.

Guided: 10–25 minutes

Example

student@server: ~/oslab-work/lab4

```
$ printf 'first observation\n' > evidence/practice.txt
$ printf 'second observation\n' >> evidence/practice.txt
$ cat evidence/practice.txt
```

- Which operator preserves the first line?

Display and save

Example

student@server: ~/oslab-work/lab4

```
$ printf 'shown and saved\n' | tee evidence/tee-demo.txt
$ printf 'one more line\n' | tee -a evidence/tee-demo.txt
```

- tee writes to the screen and a file.
- tee -a preserves earlier file contents.

Separate output from errors

Example

student@server: ~/oslab-work/lab4

```
$ cat data/not-present.csv > evidence/demo-out.txt 2> evidence/demo-error.txt
$ echo $?
$ cat evidence/demo-out.txt
$ cat evidence/demo-error.txt
```

- Inspect both files. Expected: empty stdout, diagnostic on stderr.
- Read exit status immediately; the next command changes it.

Save both streams in one file

Example

student@server: ~/oslab-work/lab4

```
$ cat data/events.csv data/not-present.csv > evidence/both.txt 2>&1
$ cat evidence/both.txt
```

- First send stdout to the file. Then 2>&1 sends stderr to the same destination.
- Inspect the file for both CSV records and a diagnostic.

Prediction: 25–35 minutes

Example

```
student@server: ~/oslab-work/lab4

$ oslab predict lab4
```

- Read your personal CSV first. Make your prediction alone with AI tools closed.
- Predict the first selected quantity, the matching-record count and the error stream.

Select, extract and count

Minutes 35 to 55. Inspect each stage before moving on.

Core 1: select, extract, count, minutes 35–55

Example

student@server: ~/oslab-work/lab4

```
$ grep '^STATUS,' data/events.csv | tee evidence/selected.txt
$ grep '^STATUS,' data/events.csv | cut -d, -f2 | tee evidence/fields.txt
$ grep '^STATUS,' data/events.csv | wc -l > evidence/normal.txt
```

- The comma excludes STATUS-late.
- Expected count: 3. Quantities are not added.

Compare and record

Example

student@server: ~/oslab-work/lab4

```
$ cat evidence/normal.txt
$ diff -u data/expected.txt evidence/normal.txt
$ nano evidence/commands.txt
$ oslab check lab4
```

- No diff output means the counts agree.
- Save the three commands you used, one per line. The helper checks your saved results.

CORE 2

Test changed and missing inputs

Minutes 55 to 70. Keep the original CSV for comparison.

Core 2: changed inputs, minutes 55–70

Example

student@server: ~/oslab-work/lab4

```
$ printf 'status,count\nOTHER,2\n' > data/no-match.csv
$ cp -- data/events.csv data/changed.csv
$ printf 'STATUS,7\nSTATUS-late,99\n' >> data/changed.csv
$ grep '^STATUS,' data/no-match.csv | wc -l > evidence/edge.txt
$ grep '^STATUS,' data/changed.csv | wc -l >> evidence/edge.txt
```

- Expected: 0 then 4. The quantity 7 adds one record.

Core 2: missing input

Example

student@server: ~/oslab-work/lab4

```
$ cat data/missing.csv > evidence/stdout.txt 2> evidence/stderr.txt
$ echo $? > evidence/status.txt
$ oslab check lab4
```

- Expected: empty stdout, diagnostic naming missing.csv, nonzero status.
- Four Core milestones check saved observations, not command authorship.
- Add your actual test commands to evidence/commands.txt.

Demonstration and optional work, minutes 70–85

Example



- The last stage may succeed even when the input command fails.
- Plus: inspect grep's no-match status (1).
- Challenge: put stderr into the pipe and use tee to display/save both streams.

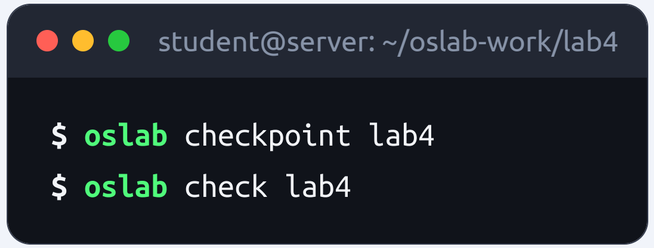
LIVE CHECKPOINT

Select from fresh records

Wait for release. Work alone with AI tools closed.

Checkpoint: 85–100 minutes

Example



1. Read the new records and submit your answers.
2. Run the printed commands to enter lab4/ and create the checkpoint CSV.
3. Save the rows for your exact status to ./evidence/checkpoint-selected.txt.
4. Save the record count to ./checkpoint.txt, beside evidence/.
5. Run the check and look for PASS on the checkpoint line.

Explain what each stage did

Use your saved observations to explain the results.

Debrief: 100–110 minutes

Example

student@server: ~/oslab-work/lab4

```
$ oslab debrief lab4
```

- Why does wc -l count records here?
- What does cut extract?
- Why is the comma important in grep?
- How did you prove a diagnostic used stderr?
- Why can a pipeline report status 0 after an input error?

Submit and grading: 110–120 minutes

- Fill in README.md from the report template and include checkpoint.txt and the required evidence files.
- Keep commands.txt and the empty stdout.txt. Commit and push with the Lab 1 Git workflow.
- Core results: 2 • commands/tests: 2 • prediction/debrief: 2 • checkpoint: 3 • submission: 1
- Scripts belong in Lab 7. Advanced extensions require later prerequisites.

EXAMPLE

More ways to process text

Optional examples for later study. Reuse the sample events.csv in ~/lab4-examples.

awk matches fields and calculates a total

Example

student@server: ~/lab4-examples

```
$ awk -F, '$1 == "ready" { print $2 }' events.csv
4
7
$ awk -F, '$1 == "ready" { sum += $2 } END { print sum+0 }' events.csv
11
```

- Use -F, for comma-separated fields. $1 is the status and $2 is the quantity.
- The first command prints the quantity for each exact ready match.
- In the second command, += adds each quantity to sum. END prints the total, 11. There are still just 2 matching records.

sort orders text or numbers

Example

student@server: ~/lab4-examples

```
$ printf '9\n12\n2\n' | sort
12
2
9
$ printf '9\n12\n2\n' | sort -n
2
9
12
```

- By default, sort compares lines as text. The line 12 starts with 1, so it comes before 2.
- sort -n compares numeric values. Now 12 comes last.

uniq collapses adjacent duplicates

Example

student@server: ~/lab4-examples

```
$ printf 'ready\n' > statuses.txt
$ printf 'done\nready\n' >> statuses.txt
$ uniq statuses.txt
ready
done
ready
$ sort statuses.txt | uniq -c
      1 done
      2 ready
```

- The first uniq leaves both ready lines because done separates them.
- sort brings matching lines together. uniq -c counts each group of equal lines.

Processes, jobs and signals

For later study after the prerequisites in the extensions handout.

Looking at processes

Command	What it shows	Output
`ps aux`	A snapshot of all processes	Can be redirected
`ps --forest`	The process tree	Can be redirected
`ps -o pid,ppid,stat,comm -p PID`	One process, with its parent and state	Can be redirected
`top, htop`	A live view, updated all the time	Interactive: take a screenshot

Start exactly one process of your own and keep its PID. Do not kill processes by name.

Jobs and signals

Signal	Number	Effect	Can be caught?
SIGTERM	15	Asks the process to end	Yes
SIGKILL	9	Ends it at once	No
SIGSTOP	19	Pauses it	No
SIGCONT	18	Resumes it	Yes
SIGINT	2	Ctrl+C from the keyboard	Yes

Example

student@server: ~/oslab-work/lab4

```
$ sleep 30 & pid=$!
$ ps -o pid,ppid,stat,comm -p "$pid"
$ kill -STOP "$pid"
$ kill -CONT "$pid"
$ kill -TERM "$pid"
$ wait "$pid"; echo "status=$?"
```

$! is the PID of the last background job. A stopped process shows state T. A wait status above 128 means a signal ended it.

Job control

Example

student@server: ~/oslab-work/lab4

```
$ sleep 60 &
$ jobs -l
$ fg %1
$ # press Ctrl+Z to pause it
$ bg %1
$ nohup sleep 60 &
```

A trailing & runs a command in the background. jobs -l lists your jobs with their PIDs.

fg %1 brings job 1 to the front. Ctrl+Z pauses the front job, and bg %1 lets it run in the background.

nohup keeps a command running after the terminal closes. Use these only on your own, short commands.

Orphans and zombies

	Orphan	Zombie
Who ends first?	The parent	The child
Is the child still running?	Yes	No, it has exited
State in ps	S or R, normal	Z, shown as defunct
Who cleans it up?	The system gives it a new parent, often PID 1	Its parent must call wait
Can you kill it?	Yes, it is a normal process	No, it is already dead: fix the parent

Run these demonstrations for a few seconds, in a disposable environment, and leave nothing running.

Key takeaways

01

Three streams

stdin, stdout and stderr are separate. Redirect each on purpose.

02

Pipes

A pipe carries only stdout. Build a pipeline one stage at a time.

03

Count records

Use wc -l to count selected rows and cut to inspect their fields.

04

Test the edges

Try inputs with no matches, an extra matching row and a missing file.

Next session

COMING UP

Lab 5: threads, kernel workers and signals

Look at threads inside a process, the kernel's own workers, and how a signal reaches a process.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 3, Processes.